Systems Administration (ASIST)

Practical Theoretical Class 10

Linux – network services through INETD
Linux – access control (*libwrap*)
Windows Server – Network Policy and Access Services
Windows Server - Hyper-V
Windows Server - Emergency Management Services
Windows Server – DCDIAG

Linux – *Standalone Server Daemons*

In order to provide a network service, a permanently available server application must be kept running to receive client requests through the protocol and port number established for the purpose.

Background applications in operating systems are commonly known as ***daemons***, many of which are network server applications.

This type of configuration makes perfect sense for high-utilization network services for which the shortest possible response time is desirable.

It is true that an under-requested service does not consume CPU resources because the process is normally ***idle*** waiting for a contact from a client, but consumes memory.

For low-utilization network services for which response time is not critical, ***inetd*** offers an alternative that avoids the need for a running process for each network service.

Linux - *Internet Service Daemon* (inetd)

The ***inetd*** network service allows, with a single running process, to replace multiple ***daemons*** while ***idle*** waiting for a client to contact.

To achieve this goal, **inetd** creates multiple UDP and TCP sockets and associates them with the port numbers corresponding to the services it provides.

Once contacted by a client in one of these sockets it creates a child process (*fork()*) and is replaced (*exec()*) with the corresponding server application..

It is the server application that will effectively dialogue with the client and not **inetd.**

Before the child process is replaced by the actual server application process, the socket descriptor is duplicated for descriptors 0 (*stdin*), 1 (*stdout*) and 2 (*stderr*).

This means that when the server application reads and writes to these descriptors, it will actually be reading from socket (*stdin*) and writing to socket (*stdout* and *stderr*).

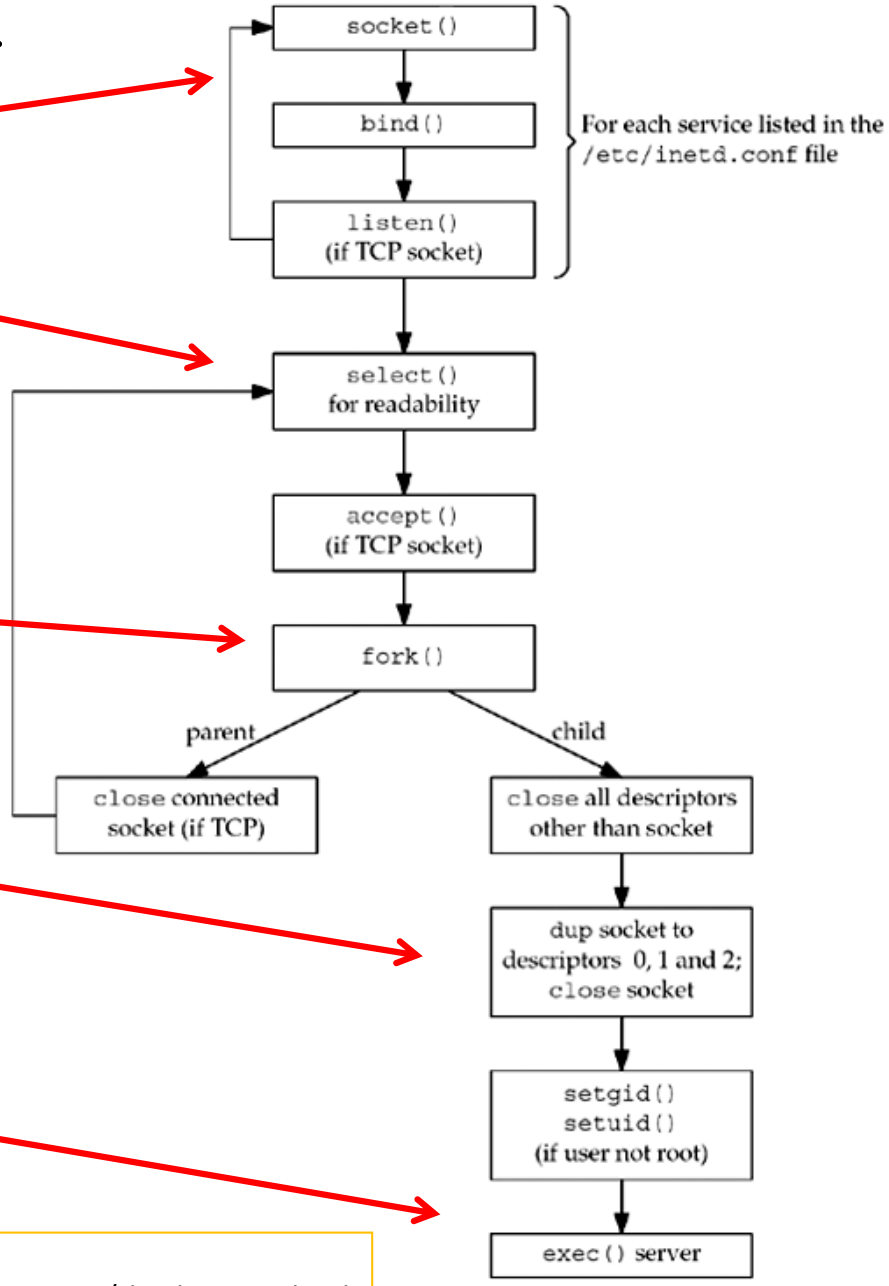
Linux – Operation of inetd

After initializing the various *sockets* INETD uses *system-call* select to detect activity on any of them..

When a client contacts one of the *sockets*, a child process is created in INETD

In the child process the descriptors *stdin*, *stdout* and *stderr* are redirected to the network connection (*socket*).

The executable is loaded in an environment where *outputs* are sent from the *socket* and *inputs* are read from the *socket*..



Source:
http://www.masterraghu.com/subjects/np/introduction/unix_network_programming_v1.3/ch13lev1sec5.html

Linux – inetd Configuration (/etc/inetd.conf)

The **/etc/inetd.conf** file defines the services provided, it is a text file in which each line defines a service, in the form:

service-name	socket-type	protocol	wait/nowait	username	exec-file	args

service-name – is the name of the service defined in the **/etc/services** file, where the protocol (UDP or TCP) and the port number are associated with the name.

socket-type – *dgram* for UDP protocol or *stream* for TCP protocol.

protocol – name of the transport protocol defined in the /**etc/protocols** file, usually UDP or TCP.

wait/nowait – ***wait*** behavior forces the server to terminate one client's service before servicing the following (typical behavior for UDP services), ***nowait*** allows serving multiple clients simultaneously (typical behavior for TCP services).

username – username under which the service will be run. Services that do not require special permissions should be performed by a user with few permissions, such as *nobody*.

service-name	socket-type	protocol	wait/nowait	username	exec-file	args

exec-file – absolute path of executable file that will be used to dialog with client according to application protocol.

args – arguments to give to the executable, including arg [0].

Example of /etc/inetd.conf file:

```
ftp      stream  tcp     nowait   root     /usr/bin/ftpd ftpd -1
daytime  stream  tcp     nowait   root     internal
telnet   stream  tcp     nowait   root     /usr/bin/telnetd telnetd
nrpe     stream  tcp     nowait   nagios   /usr/bin/nrpe  nrpe -c /etc/nrpe.cfg --inetd
tftp     dgram   udp     wait     nobody   /usr/bin/tftpd tftpd -s /tftpboot
```

In this example we can see that the second service is provided autonomously by INETD (*internal*) without resorting to any external programs.

Service names (on the left) must be defined in the system (**/etc/services**), each corresponds to a standard port number for the corresponding network service.

Linux – Network service access control

Unix family operating systems include a shared library that can be used by server applications for access control to their services, this is **libwrap.**

We can see if an application supports this library by listing all shared objects that its executable uses using the **ldd** *(List Dynamic Dependencies)* command.

For example to know if the SSH service (**/usr/sbin/sshd**) uses the **libwrap** library we can use:

```
root@myserver:~# ldd /usr/sbin/sshd
        linux-vdso.so.1 =>  (0x00007ffccedfa000)
        libwrap.so.0 => /lib/x86_64-linux-gnu/libwrap.so.0 (0x00007f22bdfa7000)
        libaudit.so.1 => /lib/x86_64-linux-gnu/libaudit.so.1 (0x00007f22bdd80000)
        libpam.so.0 => /lib/x86_64-linux-gnu/libpam.so.0 (0x00007f22bdb71000)
        libselinux.so.1 => /lib/x86_64-linux-gnu/libselinux.so.1 (0x00007f22bd94f000)
        libsystemd.so.0 => /lib/x86_64-linux-gnu/libsystemd.so.0 (0x00007f22bd8ca000)
        (…)
```

In this case it is concluded that the SSH service uses libwrap.

Linux – libwrap - configuration

Libwrap uses two files to determine whether access to a service should be allowed or not:

- **/etc/hosts.allow**
- **/etc/hosts.deny**

First the file **/etc/hosts.allow** is consulted, in case of correspondence, access is immediately allowed.

If there is no match in **/etc/hosts.allow**, the file **/etc/hosts.deny** will be consulted, in case of correspondence, access is immediately denied.

If there is no match in any of the files access is allowed.

The */etc/hosts.allow* and */etc/hosts.deny* files consist of line sequences in general form.:

DAEMONS-LIST : CLIENTS-LIST

For each access attempt, if there is a match on one line, processing ends and the following lines are ignored.

Both lists consist of value enumerations separated by spaces or commas. For an access to match a line, it must have a match on both lists of the line at the same time.

DAEMONS-LIST - define a list of process names (argv [0]). If the network service is provided by a process with one of these names, it will meet this criterion.

CLIENTS-LIST - defines a list of client addresses, can be specified in the form of IP addresses, DNS names or patterns. If the access source address matches one of the elements in this list, it will match this criterion.

In both lists the **ALL** value corresponds to any *daemon* and any client addresses respectively. There are several advanced options, for example through the **identd** network service it is possible to specify the username on the client machine as a *daemons* access criterion.

Linux – libwrap support in inetd

Current versions of INETD support libwrap:

```
root@myserver:~# ldd /usr/sbin/inetd
        linux-vdso.so.1 =>  (0x00007ffe831fb000)
        libwrap.so.0 => /lib/x86_64-linux-gnu/libwrap.so.0 (0x00007fcea44cd000)
        libbsd.so.0 => /lib/x86_64-linux-gnu/libbsd.so.0 (0x00007fcea42b8000)
        (…)
```

If INETD does not support libwrap, it can still be used through **tcpd**, in which case INETD runs tcpd which in turn runs the server application.

Examples:

```
ftp      stream  tcp     nowait   root     /usr/sbin/tcpd  /usr/bin/ftpd -1
telnet   stream  tcp     nowait   root     /usr/sbin/tcpd  /usr/bin/telnetd
ntalk    dgram   udp     wait     root     /usr/sbin/tcpd  /usr/local/lib/ntalkd
```

As you can see (example ntalk), despite its name, tcpd can also be used for UDP network services.

Windows Server – Network Policy and Access Services

Windows Server allows multiples validation types to be performed for access through the network.

The *Network Policy Server* (**NPS**) role is an implementation of the *Remote Authentication Dial-In User Service* (**RADIUS**).

RADIUS server clients are network services that interact with users and give the server authentication credentials along with service identification, the RADIUS server has as its mission:

1° - validate user credentials (***authentication***).

2° - check if the user is authorized to access the service identified (***authorization***).

3° - if access is granted, it informs the service that access should be granted and registers it (***accounting***).

Windows Server – Network Access Protection (NAP)

In Windows Server NAP is a set of mechanisms that allows you to perform various types of validation for network access.

The client machine must have an agent *NAP-capable computer* (**NAP**) that directly provides the network service (*NAP Enforcement Server*) with information about its internal state and thus the service can determine whether or not to allow access.

The declaration of state issued by the NAP agent (**SoH** – *Statement of Health*) agent typically includes: firewall state, network settings, security update and antivirus status, among others.

SoH analysis can be centralized, in which case the network service sends it to a *Network Policy Server* (**NPS**) configured to support NAP, which thus functions as a ***NAP health policy server***.

Several network services can operate as *NAP Enforcement Server*, including DHCP servers and VPN servers..

Windows Server – *NAP remediation servers*

When a client machine is declared ***noncompliant*** by NAP, access to the NAP Enforcement Server is denied.

In NPS NAP configurations it is possible to establish a ***remediation server group*** to be provided to *noncompliant* clients.

Remediation server group is a list of servers (IP addresses or DNS names) accessible to *noncompliant* clients that they should use to resolve their NAP issues and become compliant:

- Antivirus signature servers.
- Update Servers (WSUS).
- *Domain Controllers* (issues related to lack of authentication)
- *DNS/DHCP Servers* (if the client is not using the correct DNS or DHCP server).
- *Troubleshooting servers* (URLs) – pages with instructions for client administrator to make computer *compliant*.

Windows Server - Hyper-V

Hyper-V is a virtualization platform that replaces previous Microsoft equivalent systems such as Microsoft Virtual Server, Microsoft Virtual PC, and Windows Virtual PC.

Hyper-V can be enabled on recent versions of Windows, on server versions it is available as a role.

The list of operating systems supported by virtual machines (*guest* OS) includes numerous versions of Windows (server and workstation) and Unix family operating systems, including Linux.

Creating a virtual machine involves defining its hardware characteristics and the *guest* operating system you want to install, including: amount of RAM, disks you will have (virtual disks) and network interfaces (associated with virtual switches defined in Hyper-V).

Usually the *guest* operating system installation is done from a virtual CD/DVD player associated with a file with its image (.iso).

Windows Server - Hyper-V – Virtual *switches*

Virtual machine network interfaces are linked to virtual switches defined in Hyper-V. A Hyper-V virtual switch can be:

- ***External switch*** – is associated with a Windows Server physical network interface where Hyper-V is installed. Allows communication with the outside through that physical interface. This includes communicating with the server itself, other virtual machines on the same server, and virtual machines on other servers (as long as they are attached to an *external switch*). Virtual machines can directly use the physical network (*bridge*) or can share network access defined in the physical interface (NAT) settings. It is also supported the use of VLANs.
- ***Internal switch*** – only allow communications within the server, including the server itself and other virtual machines on the same server (attached to the *internal switch*).
- ***Private switch*** – defines an isolated network, only allows communication between (local) virtual machines connected to the same private switch.

Windows Server - Hyper-V – *Integration Services*

To ensure better integration between the operating system used on the virtual machine (*guest OS*) and Hyper-V, there are software packages called *Integration Services* (**hv_utils**) that can be installed on the *guest OS*.

Integration Services are made available by Hyper-V to the *guest OS* via the *Insert Integration Services Setup Disk* option which will provide the CD/DVD player with the appropriate software for the operating system concerned.

Integration Services includes support for:

- **Shutdown** *guest OS* shutdown via Hyper-V.
- *Guest OS* **Clock sync** with the server clock.
- **Access to *guest OS* state data** from Hyper-V.
- ***Heartbeat*** – guest OS monitoring by Hyper-V (*watchdog*).
- ***Backup*** (*volume snapshot*) without stopping the virtual machine.
- ***Guest services*** – allow Hyper-V to directly copy objects to *guest OS* file systems.

Windows Server – EMS (*Emergency Management Services)*

EMS is a last resort tool that can be used to access a Windows Server that for some reason is inaccessible to the administrator because it is not working properly.

For this access path to be available it must be enabled in the operating system boot options (boot volume BOOT.INI file) this file must be edited indirectly via the **bootcfg** command (note however that this command assumes that the file is *C:\BOOT.INI* which may not be true).

Although the EMS service may be configured to be accessible over the network, because in these situations the network may not be available, it is generally preferred to use a serial port, for example:

bootcfg /ems on /port com1 /baud 19200 /id N

Where **N** represents the boot entry number in the file C:\BOOT.INI. In this example, if the server becomes inaccessible by any other way, a terminal can be connected to the COM1 (argument of the command) serial port to interact via a console.

Windows Server Domain Controller - dcdiag

dcdiag is a command line tool that allows you to perform tests on the functioning of a *Domain Controller* integrated in an *Active Directory* domain.

If no arguments are given, it performs a base test set, but can be used to perform additional tests oriented to specific services, for example:

dcdiag /test:DNS

The previous command line allows you to perform various tests on the configuration and operation of the DNS service (critical in the context of an *Active Directory* domain).

This tool is a valuable helper for your administrator to quickly identify configuration and consistency issues in an *Active Directory* domain.